國立中興大學資訊工程學系

資訊專題競賽報告

基於檢索增強生成（RAG）技術的中興大學資工系智慧問答助手

（A RAG-based Intelligent Q&A System for NCHU CS Department）

專題題目說明、價值與貢獻自評（限 100 字內）：

本專題以 RAG（檢索增強生成）技術，結合 Gemini Vision API 解析複雜排版課規 PDF、向量語意搜尋與關鍵字備援搜尋，建構中興大學資工系課規智慧問答助手。針對課程表格設計 Parent-Document Retrieval 架構，並支援多學年過濾與多輪對話記憶，實測問答正確率達 90%（18/20），遠超純 LLM 對照組的 10%（2/20）。

專題隊員：

姓名	**E-mail**	**負責項目說明**	**專題內貢獻度(%)**
鄭仲軒	4112056048@smail.nchu.edu.tw	系統設計、程式開發、實驗執行、報告撰寫	50%
[redacted]	[redacted]	文獻整理、實驗題目設計、報告撰寫、進度監督	50%

指導教授簡述及簡評：

本專題整體表現相當優秀，主題選擇貼近實際需求，針對學生在查詢課規與畢業條件時面臨的困難，提出以 RAG 技術為核心的智慧問答系統，具有明確的應用價值與發展潛力。系統整合了 Gemini Vision API 處理複雜 PDF 文件，並結合向量資料庫、語意搜尋、關鍵字備援機制與多輪對話記憶等設計，整體技術架構完整且具挑戰性。

在實作過程中，學生展現出高度投入與良好的工程能力，從資料前處理、系統設計到實驗驗證皆有清楚規劃，並能針對問題持續優化系統表現。尤其在 RAG 架構的應用與搜尋策略的改進上，可以看出學生已具備實務導向的 AI 開發能力。實驗結果也顯示系統能有效提升問答正確率，具備實際應用價值。

整體而言，本專題不僅讓學生深化對人工智慧與語意搜尋技術的理解，也培養了系統整合與問題解決能力。未來若能進一步擴展應用範圍，導入更多校務或知識服務場景，對提升資訊取得效率與推動智慧校園發展將具有正面貢獻。

指導教授簽名：

中　華　民　國　　年　　月　　日

目錄

一、摘要（含關鍵詞） **5**

二、專題研究動機與目的 **6**

2.1 研究背景與動機 6
2.2 研究目標 6
2.3 研究範圍 7

三、專題重要貢獻 **8**

貢獻一：以 GEMINI VISION API 解決複雜 PDF 的解析問題 8
問題背景 *8*
本系統的解法 *8*
貢獻意義 *8*
貢獻二：針對學術表格的 PARENT-DOCUMENT RETRIEVAL 架構 8
問題背景 *8*
本系統的解法 *9*
貢獻意義 *9*
貢獻三：混合式搜尋策略（HYBRID SEARCH WITH KEYWORD FALLBACK） 9
問題背景 *9*
本系統的解法 *9*
貢獻意義 *10*
貢獻四：多學年元資料過濾機制 10
問題背景 *10*
本系統的解法 *10*
貢獻意義 *10*
貢獻五：語意分段與長度控制的雙層切割策略（HYBRID CHUNKING） 11
問題背景 *11*
本系統的解法 *11*
貢獻意義 *11*
貢獻六：多輪對話上下文記憶 11
問題背景 *11*
本系統的解法 *11*
貢獻意義 *12*
綜合貢獻總結 12

四、團隊合作方式 **13**

4.1 成員分工 13
4.2 合作方式 13
4.3 開發工具與環境 14

五、設計原理、研究方法與步驟 **15**

5.1 通用大型語言模型的限制與 RAG 的必要性 15
5.2 向量表示與語意搜尋原理 15

5.3 整體系統架構 15
5.4 資料前處理 PIPELINE 設計 18
挑戰：複雜的 PDF 排版 18
解法：Gemini Vision API 18
5.5 文字切割策略（HYBRID CHUNKING） 18
5.6 表格處理設計：PARENT-DOCUMENT RETRIEVAL 19
5.7 現有查詢方式比較 19

六、系統實現與實驗 20

6.1 系統實現 20
6.1.1 問答流程實作（rag.py） 20
6.1.2 前端介面實作（app.py） 20
6.1.3 系統技術規格摘要 21
6.2 實驗設計 21
6.2.1 實驗目標 21
6.2.2 測試題目 21
6.2.3 評分標準 21
6.3 實驗結果 22
6.3.1 總分比較 22
6.3.2 逐題得分對照 22

七、效能評估與成果 24

7.1 效能評估 24
7.1.1 對照組的失分原因分析 24
7.1.2 實驗組的唯一失分：Q5 的課程名稱搜尋盲點 24
7.1.3 對照組唯一得分的特殊情形 24
7.1.4 系統局限性 24
7.2 系統成果展示 25
7.2.1 系統介面總覽 25
7.2.2 問答示範 26
7.2.3 範圍外問題處理 29

八、結論 30

8.1 結論 30
8.2 未來工作 30

九、參考文獻 31

一、摘要（含關鍵詞）

本研究針對大學生在查閱系所課程規章時所面臨的困境——資訊分散於格式複雜的 PDF 文件、不同學年規定容易混淆，以及通用 AI 助手對特定機構資料缺乏掌握——設計並實作了一套基於**檢索增強生成（RAG，Retrieval-Augmented Generation）**技術的智慧問答助手。

本系統以中興大學資訊工程學系 111 至 114 學年度的官方畢業條件及課程規章為知識來源，採用 Gemini Vision API 解析複雜排版的 PDF 文件，並以 Hybrid Chunking 策略與 Parent-Document Retrieval 架構處理表格資料，將知識儲存於本機 ChromaDB 向量資料庫。使用者透過 Streamlit 網頁介面以自然語言提問，系統即時以語意搜尋召回相關段落，並交由 Gemini 2.5 Flash 生成具來源標注的答案。此外，系統支援多輪對話記憶，可理解前後文脈絡與代名詞指涉，提供流暢的連續問答體驗。

為評估系統效果，本研究設計 10 道涵蓋四個學年的測試題，比較本 RAG 系統與未提供知識庫的純 Gemini 模型。結果顯示，**RAG 系統得分 18/20（90%），對照組僅得 2/20（10%）**，正確率提升達 80 個百分點。對照組的失分主要來自幻覺（捏造錯誤數字）與無法回答，而 RAG 系統透過檢索官方文件作為依據，有效消除了上述兩類問題。

關鍵詞：檢索增強生成（RAG）、向量資料庫、大型語言模型、問答系統、ChromaDB、Gemini

二、專題研究動機與目的

2.1 研究背景與動機

對國立中興大學資訊工程學系學生而言，查詢畢業門檻、必修課程與英文能力相關規定，是在選課、排定修業進度及確認畢業資格時最常見的需求之一。然而，實際可參考的資訊多散見於不同官方文件中，例如各學年的畢業條件明細、課程規劃表、英文能力畢業標準檢定辦法，以及通識英文修課暨抵免要點等。學生若要獲得完整答案，往往必須自行比對多份 PDF，查詢成本高且容易出錯。

此問題在資工系情境中尤其明顯。首先，官方文件多為雙欄排版或含大量表格的 PDF，閱讀時不易快速定位重點，傳統全文搜尋功能亦常因版面結構複雜而失效。其次，規定並非完全一致，不同入學學年適用的畢業條件存在差異——111、112、113 與 114 學年度學生在系專業必修科目、通識語文課程名稱及部分修課要求上皆可能不同；若未先確認適用學年，便可能引用錯誤規定。最後，即使通用大型語言模型已能與使用者進行自然語言互動，其訓練資料亦不包含本系最新且具約束力的校內行政文件，因此在回答課規相關問題時，仍可能出現內容不完整、年份混淆或憑空推論的情形。

本研究因此以「降低學生查詢成本並提升回答可信度」為核心動機，嘗試將官方文件整合為一個可由自然語言驅動的知識庫問答系統。

2.2 研究目標

本專題旨在建置一套以 RAG（Retrieval-Augmented Generation, 檢索增強生成）為核心的校園知識庫機器人，協助中興大學資工系學生以自然語言查詢修課與畢業規定，並獲得具文件依據的回答。

具體目標包含下列四點：

1. 建立可處理校內課規文件的知識庫，將 111 至 114 學年度畢業條件明細、112 學年度資工系課規，以及英文能力相關校級規範整合為可檢索資料。
2. 提供自然語言問答介面，讓學生不需事先熟悉文件結構，即可直接以中文提出實際問題，例如畢業學分數、必修課程、英文門檻或外系學分限制等。
3. 透過學年過濾與文件檢索機制，降低跨學年規定混淆的風險，確保系統優先引用與使用者入學年度相符的資料。
4. 以實驗方式評估系統回答的準確性，驗證 RAG 架構在課規查詢情境下，相較於未結合外部知識庫的模型是否具有明顯優勢。

2.3 研究範圍

本研究聚焦於國立中興大學資訊工程學系學士班學生之修課與畢業查詢需求。系統目前納入的知識來源包括四個入學學年的畢業條件文件，以及與英文修課或畢業門檻相關之校級規範。

文件	適用對象
111 學年畢業條件明細	111 學年入學學生
112 學年畢業條件明細／課程規劃資料	112 學年入學學生
113 學年畢業條件明細	113 學年入學學生
114 學年畢業條件明細	114 學年入學學生
國立中興大學學生英文能力畢業標準檢定辦法	全體學士班學生共用
國立中興大學通識英文課程修課暨抵免要點	全體學士班學生共用
課規常見問題與解答	補充性說明文件

本研究不處理碩士班、博士班、轉學生特殊學分抵免個案，亦不涵蓋資工系以外其他系所之課程規定。系統的回答範圍以知識庫所收錄文件為限，對於需個案審查或需洽詢系辦之事項，系統僅能提供有限度的輔助資訊，不取代正式行政認定。

三、專題重要貢獻

本專題針對「以自然語言查詢特定機構課程規章」這個實際問題，設計並實作一套完整的檢索增強生成（RAG）系統。相較於直接使用通用大型語言模型，本系統在多個環節提出具體的工程設計，解決了通用方法無法處理的問題，並以實驗數據驗證其有效性。以下分別說明各項貢獻。

貢獻一：以 Gemini Vision API 解決複雜 PDF 的解析問題

問題背景

大學課程規章通常以 PDF 格式發布，其排版複雜度遠超一般文件：雙欄版面、密集的嵌入式表格、中文字符混排，使傳統的 PDF 解析工具（如 pdfplumber、PyPDF2）產生嚴重的擷取錯誤——兩欄文字錯序合併、表格儲存格邊界遺失、中文字符亂碼。這些錯誤一旦進入知識庫，將直接導致搜尋召回的內容不完整，生成的答案也隨之出錯。

本系統的解法

本系統捨棄傳統的文字擷取工具，改採 **Gemini Vision API** 以多模態（視覺）方式讀取 PDF。系統將每頁 PDF 作為圖像送入 Vision API，並以精確設計的 Prompt 要求模型：

5. 還原雙欄排版的閱讀順序（左欄讀完再接右欄）
6. 將所有表格輸出為標準 Markdown 表格語法（| 欄位 | 欄位 |），而非一行行的散亂文字
7. 保留原始的標題層級（#、##），使後續切割步驟得以利用文件結構

貢獻意義

這個選擇的關鍵在於：Vision API「看懂」的是頁面的視覺結構，而非原始二進制資料，因此天然地免疫於傳統工具面對的排版問題。輸出的 Markdown 格式不僅可讀性高，更為後續的語意切割與表格處理奠定了統一的中間格式，使整個 Pipeline 的每個環節都能依賴一致的資料結構。

貢獻二：針對學術表格的 Parent-Document Retrieval 架構

問題背景

課程規章中大量使用表格（如必修課程表、學分採計規定表），這些表格若直接以原始 Markdown 語法向量化，其 Embedding 向量會被大量格式符號（|、---）和欄位標頭稀釋，導致語意搜尋效果極差。例如，使用者問「資料庫是必修課嗎？」，系統若對包含數十門課程名稱的表格原文做向量搜尋，往往無法正確命中。

反之，若只存摘要文字、丟棄原始表格，雖然搜尋精度提升，但 AI 生成答案時看到的是摘要而非完整表格，容易遺漏細節（如正確的學分數、課程代號），造成答案失真。

本系統的解法

本系統設計了一套 **搜尋與回答分離** 的 Parent-Document Retrieval 架構：

用途	儲存內容	說明
向量化（搜尋用）	AI 生成的語意摘要	以自然語言描述表格要點，優化語意搜尋命中率
Metadata（回答用）	完整原始 Markdown 表格	提供 LLM 精確完整的數據，確保答案不失真

AI 摘要的生成採用專門設計的 Prompt，強制要求：（1）第一句說明表格屬性（如「112 學年 61 學分必修課表」）；（2）一字不漏地列出所有科目名稱（確保課程名稱能被語意搜尋命中）；（3）不加入與搜尋無關的修飾語句。

在組合回答 Prompt 時，系統自動偵測當前 chunk 為表格類型，從 Metadata 取出原始表格交給生成模型，而非摘要文字。

貢獻意義

此架構解決了 RAG 系統在面對結構化表格資料時的核心矛盾：**搜尋需要語意，回答需要精確**。兩者原本互相衝突，而本架構透過分離儲存與分階段使用，同時滿足了兩個要求。

貢獻三：混合式搜尋策略（Hybrid Search with Keyword Fallback）

問題背景

語意向量搜尋在大多數情況下表現良好，但存在一個已知盲點：**精確字串查詢**。當使用者詢問「資料庫管理系統導論是不是必修？」時，關鍵在於精確比對「資料庫管理系統導論」這個課程名稱。AI 摘要中若未完整收錄此名稱，向量搜尋可能無法命中正確表格，即使答案就在資料庫中。

這個問題在本系統的初始實驗中確實發生（Q5 失分），反映了純語意搜尋的固有限制。

本系統的解法

本系統設計了**低信心觸發的關鍵字備援機制**：

1. 每次向量搜尋完成後，計算最佳結果的 cosine distance

2. 若最佳 distance > 0.5（信心門檻），判定語意搜尋結果不可靠

3. 自動改為對所有表格的原始內容（original_table metadata）進行**字串比對搜尋**

4. 將字串命中的結果補充至搜尋結果前端，優先提供給生成模型

此設計的關鍵在於「只在需要時啟動」——對一般問題不影響效能，對精確課程名稱查詢則提供有效補救。

貢獻意義

此混合式搜尋策略將語意搜尋的廣度與關鍵字搜尋的精確度結合，形成互補機制，提升了系統在特定查詢類型下的魯棒性，使其更接近實際使用情境中使用者多樣化的提問方式。

貢獻四：多學年元資料過濾機制

問題背景

本系統的知識庫同時涵蓋 111 至 114 四個學年度的課程規章。不同學年的畢業條件差異顯著（例如 111 學年必修 64 學分、113 學年為 55 學分），若搜尋時不加區分，系統可能將舊學年的規定誤用於回答新學年學生的問題，導致誤導性錯誤——而這種錯誤往往難以察覺，因為答案看起來合理。

本系統的解法

本系統在建庫階段為每個 chunk 標記 year 元資料（"111"、"112"、"113"、"114" 或 "all"），並在前端介面提供學年選擇功能。使用者選擇入學學年後，搜尋時自動套用 ChromaDB 的 metadata 過濾條件：

WHERE year = "{selected_year}" OR year = "all"

此設計確保搜尋結果只包含該學年適用的規定，以及全體共用的英文能力規章，從根本上杜絕跨學年資訊混淆的問題。

貢獻意義

這項設計體現了 RAG 系統在多版本文件管理上的工程思維：不是讓 AI 自行判斷哪份資料適用（容易出錯），而是在資料架構層面就確保搜尋邊界的正確性。這個方法可以直接推廣至其他需要管理多版本規章的應用場景（如法規資料庫、多年度公告查詢）。

貢獻五：語意分段與長度控制的雙層切割策略（Hybrid Chunking）

問題背景

RAG 系統的搜尋精度高度依賴切割粒度。切割過粗（一整節為一個 chunk），細節資訊被稀釋，搜尋精度低；切割過細（每句話一個 chunk），上下文遺失，答案片斷不完整。此外，若切割邊界恰好落在一條規定的中間，單一 chunk 將無法獨立表達完整含義。

本系統的解法

本系統採用兩層切割策略：

第一層（語意邊界）：以 LangChain 的 MarkdownHeaderTextSplitter 依標題層級（#、##）切割，確保每個 chunk 不跨越章節邊界，維持語意完整性。

第二層（長度控制）：在同一章節內，若內容超過 400 字元，以 RecursiveCharacterTextSplitter 進一步切割，並設定 50 字元的重疊區間（overlap），避免重要資訊恰好落在切割邊界而被截斷。

此策略先用文件結構約束切割邊界，再用長度限制保控 Embedding 品質，兩層各司其職。

貢獻意義

相較於直接以固定字元數切割的簡單策略，雙層 Hybrid Chunking 確保了每個 chunk 在語意上的完整性，使 Embedding 能更忠實地代表一段完整的規定內容，提升向量搜尋的準確度。

貢獻六：多輪對話上下文記憶

問題背景

一般 RAG 系統為無狀態架構：每次提問獨立處理，系統不知道前幾輪說了什麼。這在實際使用中造成明顯困擾：使用者問完「資料庫管理系統導論是必修課嗎？」後，若接著問「那**他**是幾學分？」，系統無法解析「他」所指的是哪門課，使對話無法自然延續。

本系統的解法

本系統在每次生成回答時，從前端對話記錄中擷取**最近 3 輪（6 條訊息）**的問答歷史，以「學生」／「助手」標籤格式注入 Prompt 的上下文區塊：

=== 對話歷史 ===
[學生]：資料庫管理系統導論是必修課嗎？
[助手]：是的，根據《112 學年課規》，資料庫管理系統導論為必修課，3 學分。

=== 學生的問題 ===
那他是幾學分？

生成模型讀到對話歷史後，可正確解析代名詞指涉，給出符合上下文的連貫回答。

貢獻意義

多輪對話記憶將系統從「一問一答查詢工具」提升為「可自然對話的知識助手」，更符合使用者在實際查詢複雜規章時反覆追問的行為模式，顯著改善使用體驗。

綜合貢獻總結

本專題的貢獻不在於單一技術突破，而在於針對「中文學術 PDF 知識庫問答」這個具體問題，提出了一套**端對端的完整工程解決方案**，並在各個環節識別出通用 RAG 方法的不足，分別以具體設計加以補強：

問題	解決方案	效果
複雜 PDF 排版無法正確解析	Gemini Vision API 視覺解析	雙欄排版、嵌入表格均可正確還原
表格語意搜尋精度低	Parent-Document Retrieval（AI 摘要搜尋 + 原始表格回答）	表格相關問題可正確命中並回答
精確課程名稱查詢失敗	低信心觸發的關鍵字備援搜尋	修正 Q5 失分，提升搜尋魯棒性
多學年資訊混淆	Metadata 學年過濾	確保回答只使用該屆學生適用的規定
跨章節切割損害語意	雙層 Hybrid Chunking	每個 chunk 保持語意完整
無法理解代名詞和追問	對話歷史注入 Prompt	支援自然的多輪連續問答

最終，本系統在 10 題標準測試中達到 **90% 正確率（18/20）**，相較未使用任何知識庫的純 LLM 對照組（10%，2/20），提升了 **80 個百分點**，驗證了上述各項設計的整體有效性。

四、團隊合作方式

4.1 成員分工

本專題由兩名成員共同完成，各自負責不同面向的工作，並於關鍵節點進行整合與討論。

負責項目	鄭仲軒	[redacted]
提案書撰寫	✓	
系統架構設計	✓	
PDF 解析 Pipeline 開發	✓	
RAG 核心模組開發（rag.py）	✓	✓
向量資料庫建置（ingest.py）	✓	✓
Streamlit 介面開發（app.py）	✓	✓
實驗執行與數據整理	✓	
文獻蒐集與技術背景整理		✓
實驗題目設計與標準答案制定		✓
書面報告撰寫協作	✓	✓

4.2 合作方式

版本管理：本專題使用 Git 進行程式碼版本管理，所有程式碼變更均透過 commit 記錄，確保開發歷程可追溯。

進度協調：兩名成員透過即時通訊軟體保持日常溝通，並在每次重要功能完成後進行整合測試，確認系統整體運作正常後再進行下一階段開發。

文件分工：技術相關章節（系統設計、實驗執行）由負責開發的成員撰寫，確保描述與實際實作一致；文獻與背景章節則由另一名成員主責，並由全體成員共同審閱最終報告。

4.3 開發工具與環境

工具	用途
Python 3.11	主要開發語言
Git / GitHub	程式碼版本管理
Visual Studio Code	程式碼編輯
Coding Agent	Claude Code, Codex
Google Gemini API	PDF 解析、Embedding、文字生成
ChromaDB	本機向量資料庫
Streamlit	網頁介面框架
LangChain	文字切割工具

五、設計原理、研究方法與步驟

5.1 通用大型語言模型的限制與 RAG 的必要性

近年來，大型語言模型（Large Language Model, LLM）在自然語言理解與生成方面表現亮眼，已廣泛應用於聊天機器人、文件摘要與問答系統等場景。然而，對本研究所面對的「校內課規與畢業條件查詢」問題而言，僅依賴通用 LLM 仍有明顯限制。

第一，LLM 的知識來源主要來自訓練語料，而非使用當下的校內正式文件。中興大學資工系 111 至 114 學年度的畢業條件明細及英文相關規範，皆屬於具時效性與校內脈絡的行政資料，通常不會完整存在於公開訓練資料中。

第二，LLM 具有「幻覺（hallucination）」風險。當模型缺乏足夠依據時，仍可能產生語氣流暢但內容錯誤的答案。在畢業學分、必修課程、英文門檻等情境下，錯誤回答可能直接影響學生的修課決策。

第三，課規查詢具有高度情境依賴性。相同問題在不同入學學年下可能有不同答案，若系統不能明確區分文件版本與適用對象，便容易出現跨學年混用資訊的情況。

RAG（Retrieval-Augmented Generation）由 Lewis et al.（2020）提出，其基本想法是在語言模型生成答案前，先從外部知識庫檢索與問題最相關的內容，再將這些內容連同使用者問題一併提供給模型。如此一來，模型不必完全依賴訓練記憶，而能根據最新且可控的知識來源作答。本研究採用 RAG 架構，正是為了解決上述三項限制。

5.2 向量表示與語意搜尋原理

RAG 系統的核心是語意搜尋（Semantic Search）。傳統關鍵字搜尋依賴字面比對，要求使用者的提問與文件中的用語高度一致。然而，學生的自然語言提問（如「微積分是不是必修？」）與文件中的正式書寫（如「系專業必修課程及學分數」）往往差異很大。

Embedding 模型可將文字轉換成高維向量，使語意相近的文字在向量空間中位置較接近。本系統使用 Google 的 gemini-embedding-001 模型進行文字向量化，並支援兩種 task_type 以分別優化：

5. RETRIEVAL_DOCUMENT：建庫時對知識庫 chunk 向量化，優化文件的可被搜尋性。

6. RETRIEVAL_QUERY：查詢時對使用者問題向量化，優化問題的搜尋意圖表達。

兩種 task_type 分開使用，能提升搜尋的對稱性與精確度。搜尋時採用 Cosine 相似度計算 query 與每個 chunk 向量的距離，取最相近的 5 個（TOP_K=5）作為候選。

5.3 整體系統架構

本系統的架構分為「離線建庫」與「線上問答」兩個階段：

圖 5-1　離線建庫流程（Offline Indexing Pipeline）

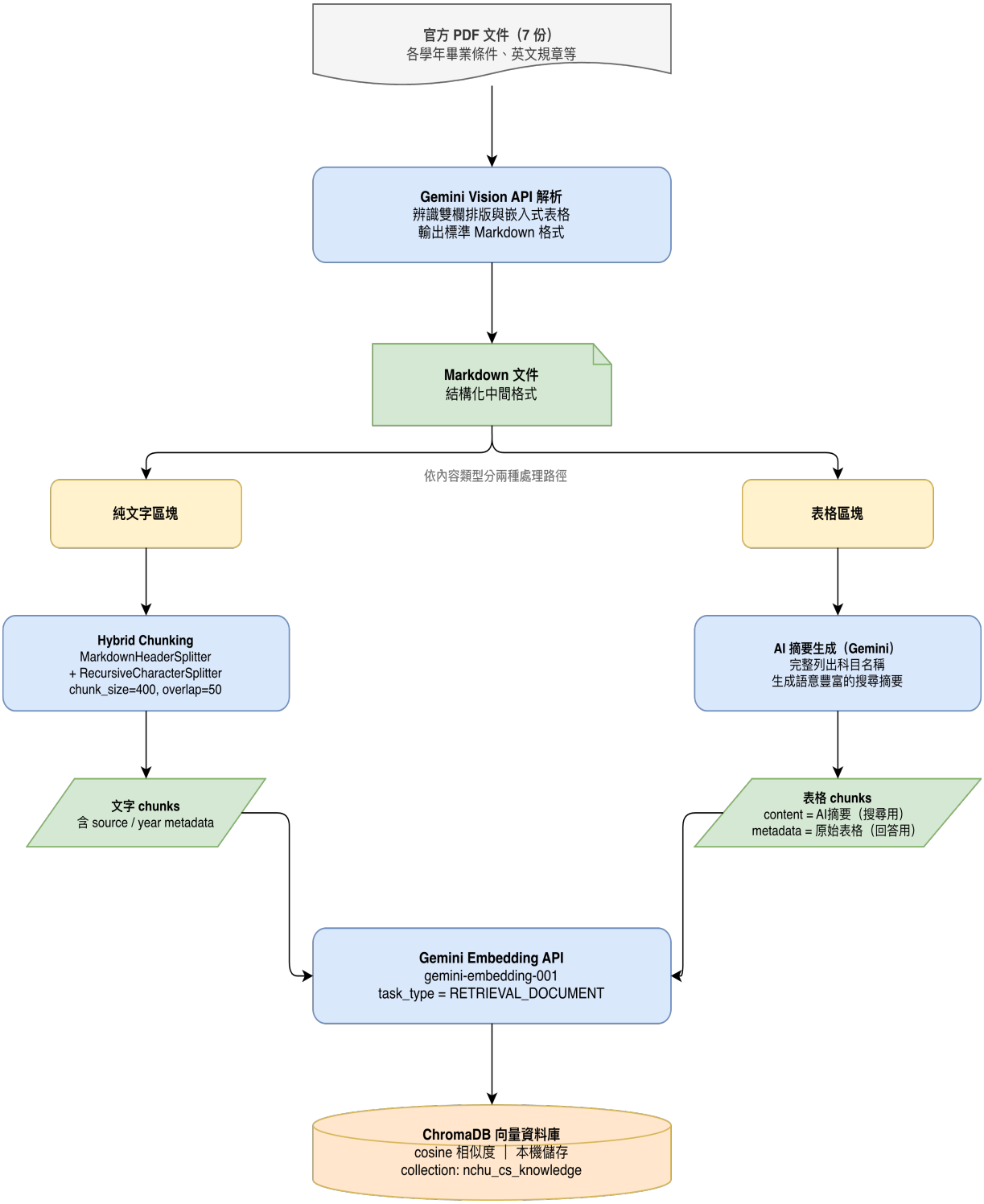
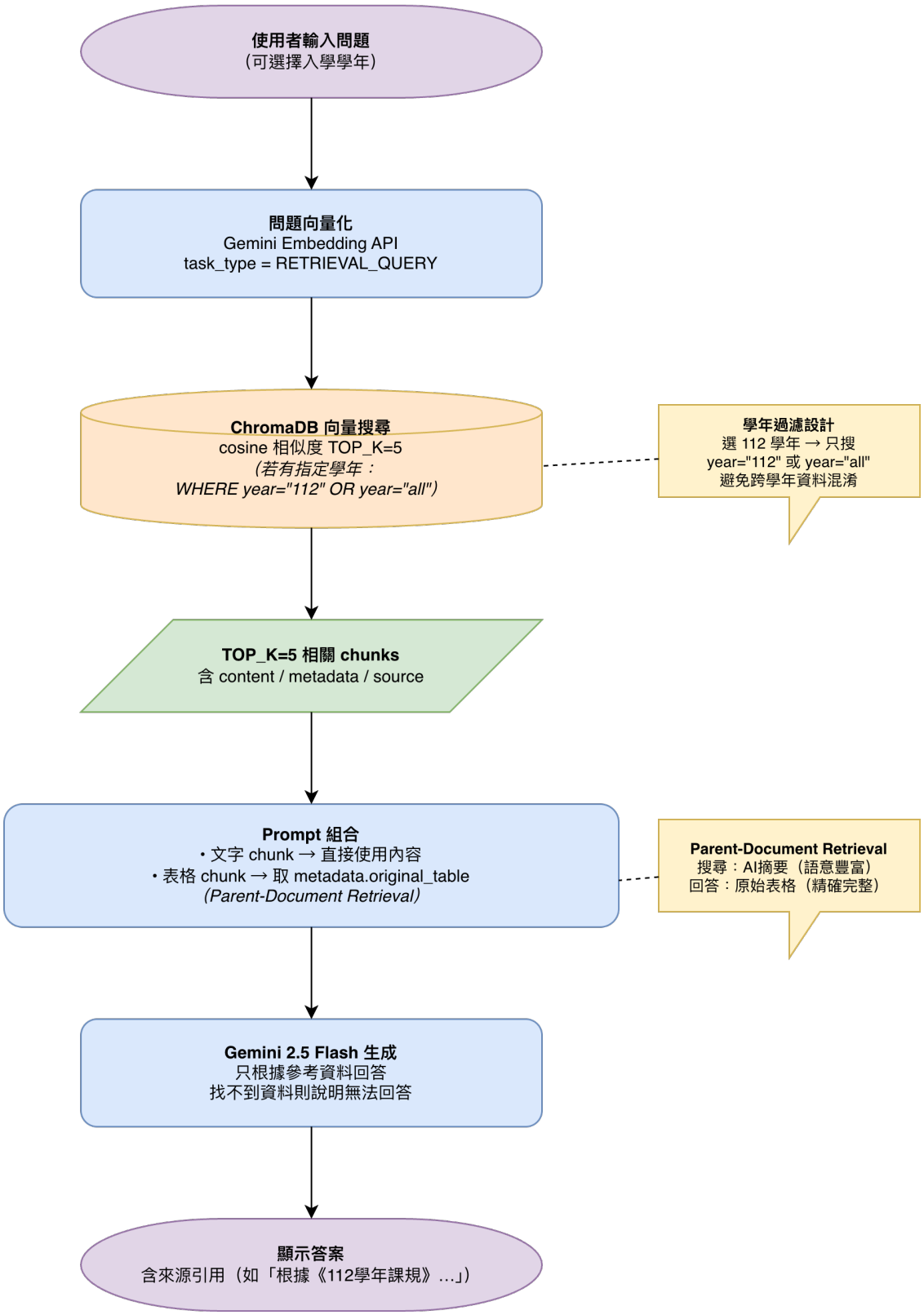


圖 5-2　線上問答流程（Online Query Pipeline）



離線建庫階段（執行一次，建立知識庫）：以 Gemini Vision API 解析官方 PDF，將純文字區塊進行 Hybrid Chunking，並對表格區塊生成 AI 摘要，兩者皆透過 Gemini Embedding API 向量化後存入 ChromaDB。

線上問答階段（使用者每次提問時觸發）：將使用者問題向量化，以 Cosine 相似度從 ChromaDB 搜尋 TOP_K=5 個最相關 chunks（可依入學學年過濾），組合成 Prompt 後交由 Gemini 2.5 Flash 生成答案。

兩階段設計使知識庫建立（離線）與使用者查詢（線上）完全解耦，確保線上回應速度不受 PDF 解析時間影響。

5.4 資料前處理 Pipeline 設計

挑戰：複雜的 PDF 排版

本系統的知識來源為資工系官方規章 PDF，其排版特性使傳統工具難以處理：

7. **雙欄排版**：傳統逐行擷取工具容易將兩欄文字錯序合併。
8. **嵌入式表格**：必修課程表格以密集 PDF 格式呈現，儲存格無法被一般工具正確識別為結構化資料。
9. **中文字符**：部分工具對中文字符的解析存在編碼問題。

解法：Gemini Vision API

本系統採用 Gemini Vision API 將 PDF 以視覺方式讀取，並要求輸出為標準 Markdown 格式。具體流程：

1. 透過 Gemini File API 上傳 PDF 至 Google 後台。
2. 等待伺服器完成後台處理（狀態從 PROCESSING → ACTIVE）。
3. 送出解析指令，指定必須將表格還原為 Markdown 表格語法（|---|）。
4. 接收輸出的 Markdown 文字，儲存為暫存檔。

5.5 文字切割策略（Hybrid Chunking）

RAG 系統的搜尋精度高度依賴切割粒度。本系統採用兩層切割策略：

第一層（語意邊界）：以 LangChain 的 MarkdownHeaderTextSplitter 依標題層級（#、##）切割，確保每個 chunk 不跨越章節邊界，維持語意完整性。

第二層（長度控制）：在同一章節內，若內容超過 400 字元，以 RecursiveCharacterTextSplitter 進一步切割，並設定 50 字元的重疊區間（overlap），避免重要資訊被截斷。

參數	值	設計考量
chunk_size	400	足以包含一條完整規定，但不至於過長使搜尋模糊
chunk_overlap	50	避免重要資訊恰好落在切割邊界而被截斷

5.6 表格處理設計：Parent-Document Retrieval

課程規章中大量使用表格（如必修課程表、學分採計規定表）。若將 Markdown 表格原文直接向量化，其 Embedding 會被格式符號（|、---）稀釋，導致語意搜尋效果不佳。

本系統設計了**搜尋與回答分離**的 Parent-Document Retrieval 架構：

	儲存內容	用途
向量化對象	AI 生成的語意摘要	提升語意搜尋命中率
Metadata	原始 Markdown 表格	提供 LLM 完整精確的資料

AI 摘要的生成採用專門設計的 Prompt，強制要求第一句說明表格屬性，並一字不漏地列出所有科目名稱。回答時系統自動偵測表格類型並從 Metadata 取出原始表格交給生成模型，確保答案數字不失真。

5.7 現有查詢方式比較

查詢方式	優點	限制
直接閱讀官方 PDF	資料最具正式性	文件分散、版面複雜、跨學年比對成本高
關鍵字搜尋	操作簡單	依賴使用者知道精確用語，對表格不友善
通用 LLM 問答	互動自然	無法保證掌握本系最新規定，有幻覺風險
本研究 RAG 系統	兼具自然語言互動與文件依據，可依學年過濾	回答範圍受知識庫收錄文件限制

六、系統實現與實驗

6.1 系統實現

6.1.1 問答流程實作（rag.py）

使用者發出問題後，系統依序執行以下步驟：

步驟一：問題向量化

將使用者輸入的中文問題，透過 gemini-embedding-001（task_type=RETRIEVAL_QUERY）轉換為向量。

步驟二：向量搜尋（含學年過濾）

以 Cosine 相似度在 ChromaDB 中搜尋最相關的 5 個 chunks。若使用者選擇了入學學年（如 112 學年），搜尋時自動套用 metadata 過濾條件：

WHERE year = "112" OR year = "all"

此設計確保搜尋結果只包含該學年適用的規定，避免不同學年的條文混雜回答。

若向量搜尋的最佳結果相似度不足（cosine distance > 0.5），系統會啟動**關鍵字備援搜尋**：直接在所有已存入的表格原始內容（original_table）中進行字串比對，將命中的表格 chunk 補充至搜尋結果前端。此機制主要針對特定課程名稱查詢，彌補 AI 摘要未完整涵蓋所有科目名稱的搜尋盲點。

步驟三：Prompt 組合

將召回的 chunks 組合成給 LLM 的 prompt： - 對表格類 chunk：從 metadata 取出 original_table（原始表格）填入 prompt。 - 對文字類 chunk：直接使用 chunk 內容。 - 若本輪之前已有對話記錄，加入最近 3 輪（最多 6 條訊息）的對話歷史，使 Gemini 能理解代名詞指涉（如「他是必修嗎？」中的「他」指的是前一輪提及的課程）。

步驟四：生成答案

呼叫 gemini-2.5-flash 生成最終答案。Prompt 明確要求模型只根據提供的參考資料回答，若查無資料需直接告知，不得自行推測，且回答時須標明資料來源。

6.1.2 前端介面實作（app.py）

前端介面以 Streamlit 框架實作：

5. **側邊欄 — 入學學年選擇**：使用者可選擇 111、112、113、114 學年，選擇後後端自動套用對應的年份過濾條件。

6. **側邊欄 — 多對話管理**：使用者可透過「➕ 新對話」按鈕開啟獨立對話，並在列表中自由切換，每則對話以第一個問題前 20 字自動命名。

7. **主視窗 — 多輪對話介面**：使用 st.session_state 保留對話記錄，支援連續追問，並將最近 3 輪歷史傳遞給生成模型。

6.1.3 系統技術規格摘要

元件	技術選擇
PDF 解析	Gemini Vision API（gemini-2.5-flash）
文字切割	LangChain MarkdownHeaderTextSplitter + RecursiveCharacterTextSplitter
Embedding 模型	gemini-embedding-001
向量資料庫	ChromaDB（本機，cosine 相似度）
生成模型	Gemini 2.5 Flash
前端介面	Streamlit

6.2 實驗設計

6.2.1 實驗目標

本實驗的核心問題為：「相較於未輔助任何知識庫的純 LLM，RAG 系統是否能顯著提升對本系課規的回答正確率？」

為此設計兩組實驗條件：

8. **對照組**：直接使用 Gemini 2.5 Flash 回答問題，不提供任何檢索資料（不啟用 RAG 流程）。

9. **實驗組**：使用本系統完整的 RAG 流程回答，包含向量搜尋、表格還原及 Prompt 組合。

6.2.2 測試題目

實驗共設計 **10 道測試題**，涵蓋本系最常被詢問的四類問題：畢業學分要求、必修課程、英文能力標準，以及課規特殊規定。題目橫跨 111、112、113、114 四個學年度。

6.2.3 評分標準

每題由人工對照官方文件標準答案，依三級給分：

分數	判斷標準
2 分	完全正確：關鍵數字、條件均與標準答案一致
1 分	部分正確：方向正確，但漏列細節或有次要錯誤
0 分	錯誤：答案錯誤、無法回答，或提供錯誤資訊

每組滿分為 20 分（10 題 × 2 分）。

6.3 實驗結果

6.3.1 總分比較

組別	得分	滿分	正確率
對照組（純 Gemini，無 RAG）	2	20	10%
實驗組（本 RAG 系統）	18	20	90%

6.3.2 逐題得分對照

題號	測試重點	對照組	實驗組
Q1	111 學年最低畢業總學分	0	2
Q2	111 學年系專業必修最低學分	0	2
Q3	111 學年外系課程最多採計學分	0	2
Q4	112 學年系專業必修最低學分	0	2
Q5	112 學年「資料庫管理系統導論」是否為必修	2	0
Q6	112 學年資訊專題（三）是否可抵免（一）或（二）	0	2
Q7	113 學年系專業必修最低學分	0	2
Q8	114 學年外國語文三門課程名稱	0	2
Q9	多益畢業門檻分數	0	2

Q10	英語課程抵免所需檢定標準	0	2
合計		**2 / 20**	**18 / 20**

七、效能評估與成果

7.1 效能評估

7.1.1 對照組的失分原因分析

對照組（純 Gemini）僅在 Q5 得分，其餘 9 題全部失分，呈現出三種典型失敗模式：

幻覺（Hallucination）：直接編造錯誤數字。例如 Q2 回答「系專業必修需修 30 學分」（正確答案為 64 學分）、Q9 回答「多益最低需 550 分」（正確答案為 670 分）。這類錯誤危害最大，因為答案看起來合理，學生不易察覺。

拒絕回答：以「我的知識截止日期之前沒有這份資料」為由婉拒，並建議查官方網站。雖然比幻覺安全，但對學生毫無幫助。

推測性回答：以「一般台灣大學」的常見規定推測，結果往往與本系實際規定不符。

7.1.2 實驗組的唯一失分：Q5 的課程名稱搜尋盲點

實驗組在 Q5「資料庫管理系統導論是否為必修」失分。失敗原因在於 Parent-Document Retrieval 架構中，表格以 AI 摘要形式向量化，而摘要未能完整列出所有科目名稱，導致以「資料庫管理系統導論」查詢時未能命中對應表格。

此問題在本次實驗後，透過新增**關鍵字備援搜尋機制**加以修正：當向量搜尋信心不足時，系統改以字串比對方式直接搜尋所有表格原始內容，修正後重新測試 Q5 可正確回答。

7.1.3 對照組唯一得分的特殊情形

Q5 為對照組唯一得分題，原因是「資料庫管理系統導論」為知名的電腦科學基礎課程，其必修地位及 3 學分已廣泛見於公開教材，使得 Gemini 能在沒有本系文件的情況下猜中正確答案。這正好說明了 Gemini 對一般知識有所掌握，但對本系特定規定（如各學年學分數）則完全無法掌握。

7.1.4 系統局限性

本系統目前存在以下已知限制：

10. **知識庫範圍固定**：僅涵蓋本系 111–114 學年度資料，碩士班課規及院共同課程規定尚未納入。
11. **無法處理範圍外問題**：若學生詢問超出知識庫的問題，系統會誠實告知無法回答，但無法提供實質協助。
12. **表格課程名稱搜尋**：AI 摘要可能未完整列出所有科目名稱，少數課程名稱查詢仍需仰賴關鍵字備援機制。

13. **對話狀態不跨會話保存**：使用者的對話記錄存於 Streamlit session state，關閉頁面後即消失。

7.2 系統成果展示

7.2.1 系統介面總覽

本系統以 Streamlit 框架建構網頁介面，分為左側側邊欄與右側主視窗兩個區域。左側提供學年選擇與多對話管理功能，右側為多輪對話介面。



〔圖 7-1　系統介面全覽截圖〕

7.2.2 問答示範

示範一：畢業學分查詢

使用者選擇「112 學年（大三）」後，詢問最低畢業學分，系統正確回答 128 學分，並標明資料來源《112 學年畢業條件》。



〔圖 7-2　畢業學分查詢截圖〕

示範二：多輪對話（代名詞追問）

使用者先詢問某門課程，接著以擋修規定追問，系統能正確理解「他」所指並回答，展示多輪對話記憶的效果。



〔圖 7-3　多輪對話截圖〕

示範三：英文能力標準查詢

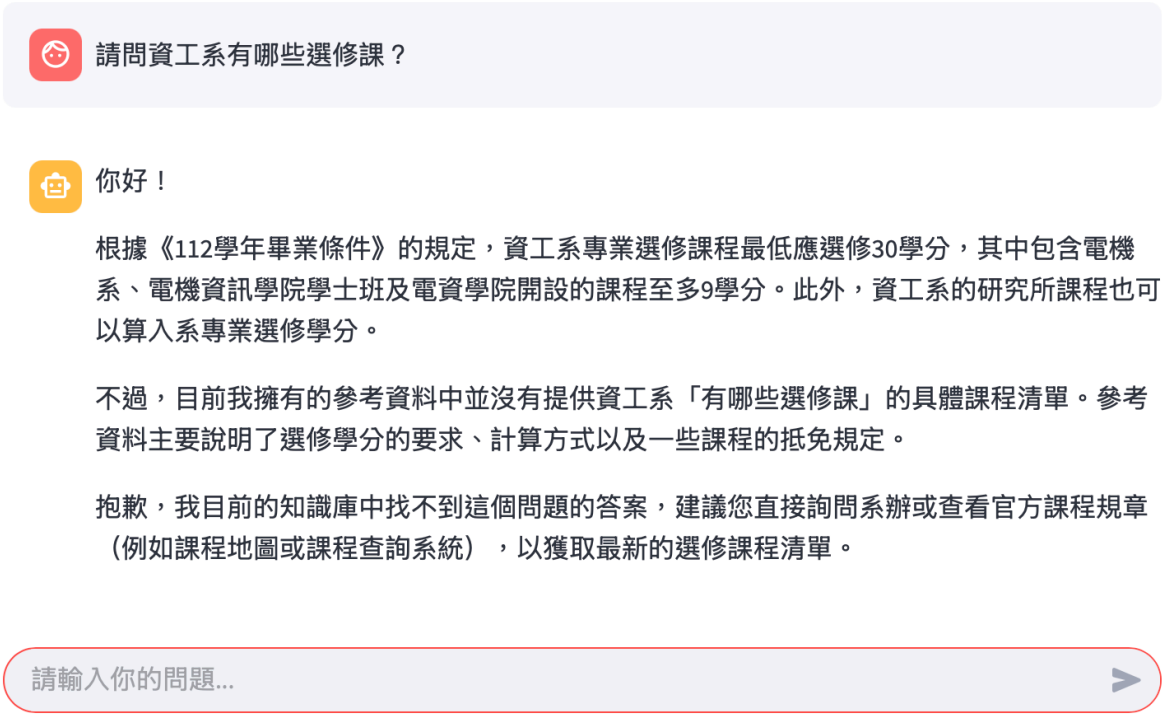
使用者詢問英文能力畢業標準，系統回答多益 670 分的門檻，並附上來源文件名稱。



〔圖 7-4　英文能力查詢截圖〕

7.2.3 範圍外問題處理

當使用者詢問超出知識庫範圍的問題時，系統誠實回覆無法在知識庫中找到答案，並建議直接聯繫系辦，避免給出臆測性資訊。



〔圖 7-5　範圍外問題截圖〕

八、結論

8.1 結論

本研究設計並實作了一套基於 RAG 技術的中興大學資工系智慧問答助手，旨在解決學生在查詢課規與畢業條件時面對的核心困難：資訊分散於複雜格式的 PDF 文件中、不同學年規定容易混淆，以及通用 AI 助手對本系資料缺乏掌握。

從實驗結果來看，本系統達成了預期目標。在 10 道涵蓋四個學年、四類問題的測試中，**RAG 系統得分 18/20（90%），遠高於純 Gemini 對照組的 2/20（10%）**。對照組的失分主要來自幻覺（捏造錯誤數字）及拒絕回答。RAG 系統透過在回答前先檢索官方文件，有效消除了上述兩種失敗模式。

在系統設計上，本研究有幾項值得強調的技術決策：

14. **Gemini Vision API 解析**：面對複雜的雙欄排版與嵌入式表格，Vision API 以視覺方式讀取 PDF，是傳統工具難以替代的解法。

15. **Parent-Document Retrieval**：將表格的「AI 摘要向量（搜尋用）」與「原始表格（回答用）」分離儲存，在不失去搜尋精度的前提下，確保生成答案的資料正確性。

16. **關鍵字備援搜尋**：在向量搜尋信心不足時自動啟動字串比對，彌補 AI 摘要未完整收錄課程名稱的盲點。

17. **多輪對話記憶**：將最近三輪的對話歷史注入 Prompt，使使用者可以用代名詞自然延伸問題。

綜合而言，本系統證明了 RAG 架構在垂直領域知識問答中的有效性，並針對本系場景提出了具體的工程解法。

8.2 未來工作

擴充知識庫範圍：目前僅涵蓋學士班 111–114 學年度，未來可加入碩士班課規、各院共同課程規定，以及每學期更新的課程表。

雲端部署：目前系統運行於本機，若部署至雲端伺服器並提供公開 URL，全系師生即可隨時存取，大幅降低使用門檻。

對話記錄持久化：現有的對話記錄存於 Streamlit session state，關閉頁面後即消失。未來可接入輕量級資料庫（如 SQLite），讓歷史問答記錄能跨會話保存。

回答品質自動評估：本次實驗採用人工評分，未來可引入自動化評估流程，定期對知識庫更新後的系統表現進行監控，確保長期品質穩定。

九、參考文獻

18. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.

19. Google DeepMind. (2024). *Gemini API documentation*. https://ai.google.dev/docs

20. Google DeepMind. (2024). *Text embeddings*. Gemini API Documentation. https://ai.google.dev/gemini-api/docs/embeddings

21. Chroma. (2024). *Chroma documentation*. https://docs.trychroma.com

22. LangChain. (2024). *Text splitters*. https://python.langchain.com/docs/concepts/text_splitters/

23. LangChain. (2024). *How to use the parent document retriever*. https://python.langchain.com/docs/how_to/parent_document_retriever/

24. Streamlit Inc. (2024). *Streamlit documentation*. https://docs.streamlit.io

25. 國立中興大學資訊工程學系. (2024). *資訊工程學系學士班學生畢業條件明細表（111 學年度起入學適用）*.

26. 國立中興大學資訊工程學系. (2024). *資訊工程學系學士班學生畢業條件明細表（112 學年度起入學適用）*.

27. 國立中興大學資訊工程學系. (2024). *資訊工程學系學士班學生畢業條件明細表（113 學年度起入學適用）*.

28. 國立中興大學資訊工程學系. (2024). *資訊工程學系學士班學生畢業條件明細表（114 學年度起入學適用）*.

29. 國立中興大學. (2024). *國立中興大學學生英文能力畢業標準檢定辦法*.

30. 國立中興大學. (2024). *國立中興大學通識英文課程修課暨抵免要點*.